

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# =====
# Neural Network with One Hidden Layer
# =====

class MLP:

    def __init__(self, inputs, targets, nHidden):

        # Number of input features
        self.nIn = inputs.shape[1]

        # Number of output neurons
        self.nOut = targets.shape[1]

        # Number of hidden neurons
        self.nHidden = nHidden

        # Input layer -> Hidden layer
        # +1 is for the bias
        self.weights1 = (
            np.random.randn(self.nIn + 1, self.nHidden)
            * 0.1

```

```
)
```

```
# Hidden layer -> Output layer
```

```
# +1 is for the bias
```

```
self.weights2 = (
```

```
    np.random.randn(self.nHidden + 1, self.nOut)
```

```
    * 0.1
```

```
)
```

```
# -----
```

```
# Sigmoid activation function
```

```
# -----
```

```
def sigmoid(self, x):
```

```
    return 1.0 / (1.0 + np.exp(-x))
```

```
# -----
```

```
# Forward propagation
```

```
# -----
```

```
def forward(self, inputs):
```

```
    # Add bias input
```

```
    # The original Marsland code uses -1 for the bias
```

```
    inputs_bias = np.concatenate(
```

```
(
    inputs,
    -np.ones((inputs.shape[0], 1))
),
axis=1
)
```

Input layer -> Hidden layer

```
hidden_activation = np.dot(
    inputs_bias,
    self.weights1
)
```

Apply sigmoid activation

```
hidden_output = self.sigmoid(
    hidden_activation
)
```

Add bias to hidden layer

```
hidden_output_bias = np.concatenate(
    (
        hidden_output,
        -np.ones((hidden_output.shape[0], 1))
    ),
    axis=1
)
```

```
# Hidden layer -> Output layer
```

```
output_activation = np.dot(
```

```
    hidden_output_bias,
```

```
    self.weights2
```

```
)
```

```
# Apply sigmoid activation
```

```
output = self.sigmoid(
```

```
    output_activation
```

```
)
```

```
return (
```

```
    inputs_bias,
```

```
    hidden_activation,
```

```
    hidden_output_bias,
```

```
    output_activation,
```

```
    output
```

```
)
```

```
# -----
```

```
# Training using backpropagation
```

```
# -----
```

```
def train(self, inputs, targets, eta, nIterations):
```

```
error_history = []
```

```
for n in range(nIterations):
```

```
    # -----
```

```
    # Forward propagation
```

```
    # -----
```

```
    (
```

```
        inputs_bias,
```

```
        hidden_activation,
```

```
        hidden_output_bias,
```

```
        output_activation,
```

```
        outputs
```

```
) = self.forward(inputs)
```

```
    # -----
```

```
    # Calculate output error
```

```
    # -----
```

```
    error = targets - outputs
```

```
    # Mean squared error
```

```
    mse = np.mean(error ** 2)
```

```
    error_history.append(mse)
```

```

# -----
# Output layer gradient
# -----

# Derivative of sigmoid:
#
#  $\text{sigmoid}'(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$ 
#
output_derivative = (
    outputs * (1 - outputs)
)

# Output delta
output_delta = (
    error
    *
    output_derivative
)

# -----
# Update hidden -> output weights
# -----

self.weights2 += eta * np.dot(
    hidden_output_bias.T,

```

```

        output_delta
    )

    # -----
    # Backpropagate error
    # -----

    # We do not use the bias weight when
    # propagating error back to hidden neurons
    hidden_error = np.dot(
        output_delta,
        self.weights2[:-1, :].T
    )

    # -----
    # Hidden layer gradient
    # -----

    hidden_output = (
        hidden_output_bias[:, :-1]
    )

    hidden_derivative = (
        hidden_output
        *
        (1 - hidden_output)
    )

```

```
)
```

```
# Hidden delta
```

```
hidden_delta = (
```

```
    hidden_error
```

```
    *
```

```
    hidden_derivative
```

```
)
```

```
# -----
```

```
# Update input -> hidden weights
```

```
# -----
```

```
self.weights1 += eta * np.dot(
```

```
    inputs_bias.T,
```

```
    hidden_delta
```

```
)
```

```
return error_history
```

```
# -----
```

```
# Prediction
```

```
# -----
```

```
def predict(self, inputs):
```



```
(
    inputs_bias,
    hidden_activation,
    hidden_output_bias,
    output_activation,
    output
) = self.forward(inputs)
```

```
# Convert probability to class
```

```
predictions = np.where(
    output >= 0.5,
    1,
    0
)
```

```
return predictions
```

```
# =====
```

```
# 1. Load Pima Indians Diabetes Dataset
```

```
# =====
```

```
# If your file is called something else,
```

```
# change "diabetes.csv" to your filename.
```

```
# Check if 'diabetes.csv' exists before trying to read it
```

```
import os
```

```

if not os.path.exists("diabetes.csv"):

    print("Error: 'diabetes.csv' not found. Please upload the file.")

    # You can add a sys.exit() here if you want to stop execution

    # or handle the error more gracefully.

    # For now, we'll let it raise FileNotFoundError if it proceeds.


data = pd.read_csv("diabetes.csv")


# Display first five rows
print("First five rows of dataset:")
print(data.head())


# =====

# 2. Separate Inputs and Targets

# =====


# The first 8 columns are input features
X = data.iloc[:, 0:8].values


# The last column is the target
y = data.iloc[:, 8:9].values


# =====

# 3. Normalize the Input Data

# =====

```

```
# Calculate mean and standard deviation
```

```
mean = X.mean(axis=0)
```

```
std = X.std(axis=0)
```

```
# Standardization
```

```
X = (X - mean) / std
```

```
# =====
```

```
# 4. Shuffle Dataset
```

```
# =====
```

```
np.random.seed(42)
```

```
indices = np.arange(len(X))
```

```
np.random.shuffle(indices)
```

```
X = X[indices]
```

```
y = y[indices]
```

```
# =====
```

```
# 5. Split Dataset into Training and Testing
```

```
# =====
```

```
split = int(0.8 * len(X))
```

```
X_train = X[:split]
```

```
y_train = y[:split]
```

```
X_test = X[split:]
```

```
y_test = y[split:]
```

```
print()
```

```
print("Training data shape:", X_train.shape)
```

```
print("Testing data shape:", X_test.shape)
```

```
# =====
```

```
# 6. Create Neural Network
```

```
# =====
```

```
# The architecture is:
```

```
#
```

```
# 8 input neurons
```

```
#    ↓
```

```
# 5 hidden neurons
```

```
#    ↓
```

```
# 1 output neuron
```

```
network = MLP(
```

```
    X_train,
```

```
    y_train,
```

```
    nHidden=5
```

```
)
```

```
# =====
```

```
# 7. Train the Neural Network
```

```
# =====
```

```
learning_rate = 0.1
```

```
number_of_iterations = 1000
```

```
error_history = network.train(
```

```
    X_train,
```

```
    y_train,
```

```
    eta=learning_rate,
```

```
    nIterations=number_of_iterations
```

```
)
```

```
# =====
```

```
# 8. Test the Network
```

```
# =====
```

```
predictions = network.predict(
```

```
    X_test
```

```
)
```

```
# =====
```

```
# 9. Calculate Accuracy
```

```
# =====
```

```
accuracy = np.mean(  
    predictions == y_test  
)
```

```
print()  
print("Accuracy:", accuracy)  
print("Accuracy percentage:", accuracy * 100, "%")
```

```
# =====
```

```
# 10. Confusion Matrix
```

```
# =====
```

```
confusion_matrix = np.zeros(  
    (2, 2),  
    dtype=int  
)
```

```
for actual, predicted in zip(  
    y_test.flatten(),  
    predictions.flatten()  
):
```

```
    confusion_matrix[  
        int(actual),
```

```
        int(predicted)
    ] += 1

print()
print("Confusion Matrix:")
print(confusion_matrix)

# =====

# 11. Plot Training Error

# =====

plt.figure(figsize=(8, 5))

plt.plot(
    error_history
)

plt.xlabel("Iteration")
plt.ylabel("Mean Squared Error")
plt.title("Training Error")

plt.show()
```

First five rows of dataset:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Training data shape: (614, 8)

Testing data shape: (154, 8)

Accuracy: 0.7597402597402597

Accuracy percentage: 75.97402597402598 %

Confusion Matrix:

[[90 6]

[31 27]]

